

Beyond the GIL: Python's Journey to Free Threading

PyCon Israel (Dry Run), August 2025

bit.ly/nogil

Yonatan Bitton
@bityob

whoami

- I'm **Yonatan Bitton**, core team @ **Fortinet**
- We provide cybersecurity protection for emails and more
- Passionate about **Python**, enjoy solving tough **challenges** and understanding things **inside out**

Agenda

Agenda

- A Quick Look Back

Agenda

- A Quick Look Back
- The Problem: Why the GIL?

Agenda

- A Quick Look Back
- The Problem: Why the GIL?
- The Solution: PEP 703

Agenda

- A Quick Look Back
- The Problem: Why the GIL?
- The Solution: PEP 703
- Challenges & The Road Ahead

Agenda

- A Quick Look Back
- The Problem: Why the GIL?
- The Solution: PEP 703
- Challenges & The Road Ahead
- Summary

A Quick Look Back

- 1991: Python 0.9.0

- 1991: Python 0.9.0
- 1992: Threads + the GIL introduced

- 1991: Python 0.9.0
- 1992: Threads + the GIL introduced
- 1996: Greg Stein's "free threading" patch

- 1991: Python 0.9.0
- 1992: Threads + the GIL introduced
- 1996: Greg Stein's "free threading" patch
- 2007: python-safethread by Adam Olsen

- 1991: Python 0.9.0
- 1992: Threads + the GIL introduced
- 1996: Greg Stein's "free threading" patch
- 2007: python-safethread by Adam Olsen
- 2016: Gilectomy (Larry Hastings)

The GIL survives.

Why the GIL?

Why the GIL?

- Global Interpreter Lock (GIL) allows only one thread to run Python code at a time

Why the GIL?

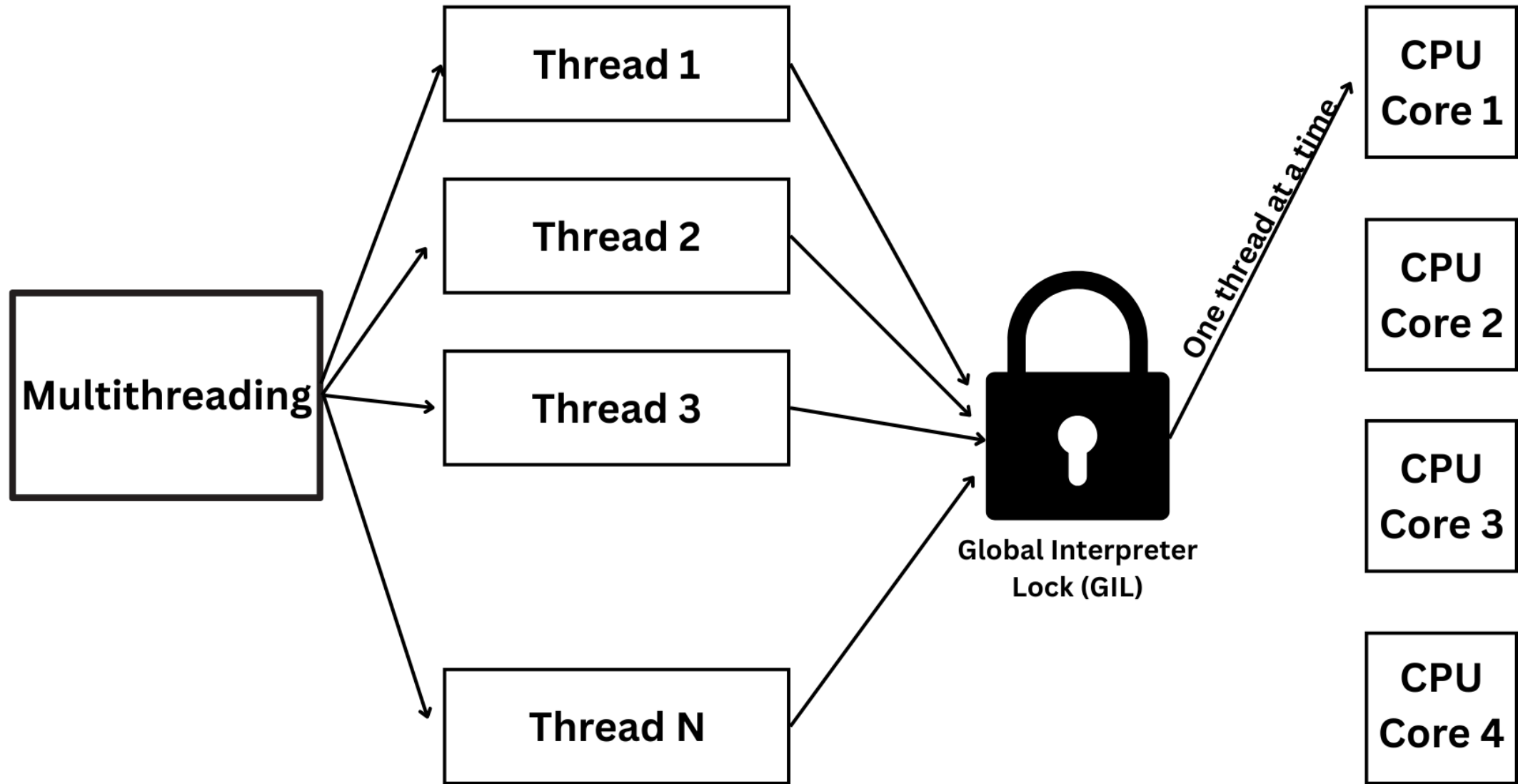
- Global Interpreter Lock (GIL) allows only one thread to run Python code at a time
- Simplified CPython's memory management

Why the GIL?

- Global Interpreter Lock (GIL) allows only one thread to run Python code at a time
- Simplified CPython's memory management
- Ensured thread safety with a single lock

Why the GIL?

- Global Interpreter Lock (GIL) allows only one thread to run Python code at a time
- Simplified CPython's memory management
- Ensured thread safety with a single lock
- Made sense in the **single-core era**



Why Not the GIL?

Why Not the GIL?

- Blocks true multi-core parallelism

Why Not the GIL?

- Blocks true multi-core parallelism
- Bottleneck for CPU-bound code

Why Not the GIL?

- Blocks true multi-core parallelism
- Bottleneck for CPU-bound code
- Wastes modern multi-core CPUs

What About Workarounds?

What About Workarounds?

- AsyncIO: concurrency, not parallelism

What About Workarounds?

- AsyncIO: concurrency, not parallelism
- C extensions: can release GIL — but risky

What About Workarounds?

- AsyncIO: concurrency, not parallelism
- C extensions: can release GIL — but risky
- Multiprocessing: works — but heavyweight

What About Workarounds?

- AsyncIO: concurrency, not parallelism
- C extensions: can release GIL — but risky
- Multiprocessing: works — but heavyweight
- Developers want **real** threads

Why Not Just Remove the GIL?

Why Not Just Remove the GIL?

- The GIL is deeply embedded in CPython's memory management

Why Not Just Remove the GIL?

- The GIL is deeply embedded in CPython's memory management
- Removing it breaks assumptions across the interpreter

Why Not Just Remove the GIL?

- The GIL is deeply embedded in CPython's memory management
- Removing it breaks assumptions across the interpreter
- Threads will race on object lifetimes, containers, and internal APIs

But What If We Could?...

- 2021: nogil (Sam Gross)

- 2021: nogil (Sam Gross)
- 2023: PEP 703 published (Jan 2023)

PEP 703 – Making the Global Interpreter Lock Optional in CPython

Author: Sam Gross <colesbury at gmail.com>

Sponsor: Łukasz Langa <lukasz at python.org>

Discussions-To: [Discourse thread](#)

Status: [Accepted](#)

Type: [Standards Track](#)

Created: 09-Jan-2023

Python-Version: 3.13

Post-History: [09-Jan-2023](#), [04-May-2023](#)

Resolution: [24-Oct-2023](#)

- 2023: Meta supports and commits three engineer-years through 2025



A fast, free threading Python

■ Ideas



Carl Meyer carljm CPython core developer

Jul 2023



Guido van Rossum:



it would be great if Meta or another tech company could spare some engineers with established CPython internals experience to help the core dev team with this work.

We've had a chance to discuss this internally with the right people. Our team believes in the value that nogil will provide, and we are committed to working collaboratively to improve Python for everyone.

If PEP 703 is accepted, Meta can commit to support in the form of three engineer-years (from engineers experienced working in CPython internals) between the acceptance of PEP 703 and the end of 2025, to collaborate with the core dev team on landing the PEP 703 implementation smoothly in CPython and on ongoing improvements to the compatibility and performance of nogil CPython.

- 2023: PEP 703 **accepted** with clear proviso (Oct 2023)

Note

The Steering Council accepts PEP 703, but with clear proviso: that the rollout be gradual and break as little as possible, and that we can roll back any changes that turn out to be too disruptive – which includes potentially rolling back all of PEP 703 entirely if necessary (however unlikely or undesirable we expect that to be).

- 2024: Python 3.13 with **experimental** free-threaded build
 - Opt-in via `--disable-gil` compile flag

- 2024: Python 3.13 with **experimental** free-threaded build
 - Opt-in via `--disable-gil` compile flag
- 2025: PEP 779: Criteria for supported status

- 2024: Python 3.13 with **experimental** free-threaded build
 - Opt-in via `--disable-gil` compile flag
- 2025: PEP 779: Criteria for supported status
- 2025: Python 3.14.0rc1 released in July 2025

- 2024: Python 3.13 with **experimental** free-threaded build
 - Opt-in via `--disable-gil` compile flag
- 2025: PEP 779: Criteria for supported status
- 2025: Python 3.14.0rc1 released in July 2025
 - Not **experimental**, not yet the default build

- 2024: Python 3.13 with **experimental** free-threaded build
 - Opt-in via `--disable-gil` compile flag
- 2025: PEP 779: Criteria for supported status
- 2025: Python 3.14.0rc1 released in July 2025
 - Not **experimental**, not yet the default build
- 2030~: Python default build?

The free-threaded mode is experimental and work is ongoing to improve it: expect some bugs and a substantial single-threaded performance hit.

How to use?

```
$ uv python install 3.14t  
Installed Python 3.14.0b4 in 12.54s  
+ cpython-3.14.0b4+freethreaded-linux-x86_64-gnu (python3.14t)
```

```
1 # demo-freethreaded-python.py
2 import time
3 from concurrent.futures import ThreadPoolExecutor
4
5 N = 50_000_000
6 THREADS = 10
7
8 def work(n):
9     for _ in range(n):
10         pass
11
12 start = time.time()
13
14 with ThreadPoolExecutor(max_workers=THREADS) as pool:
15     list(pool.map(work, [N] * THREADS))
16
```

```
3 from concurrent.futures import ThreadPoolExecutor
4
5 N = 50_000_000
6 THREADS = 10
7
8 def work(n):
9     for _ in range(n):
10         pass
11
12 start = time.time()
13
14 with ThreadPoolExecutor(max_workers=THREADS) as pool:
15     list(pool.map(work, [N] * THREADS))
16
17 print(f"Done in {time.time() - start:.2f} sec")
```

```
$ uv run --python=3.14 python demo-freethreaded-python.py  
Done in 9.39 sec
```

```
$ uv run --python=3.14t python demo-freethreaded-python.py  
Done in 0.99 sec
```

Removing the GIL: What Had to Change

- Reference Counting
- Garbage Collection
- Locking and atomic APIs
- Containers Thread-Safety

Reference Counting

Reference Counting

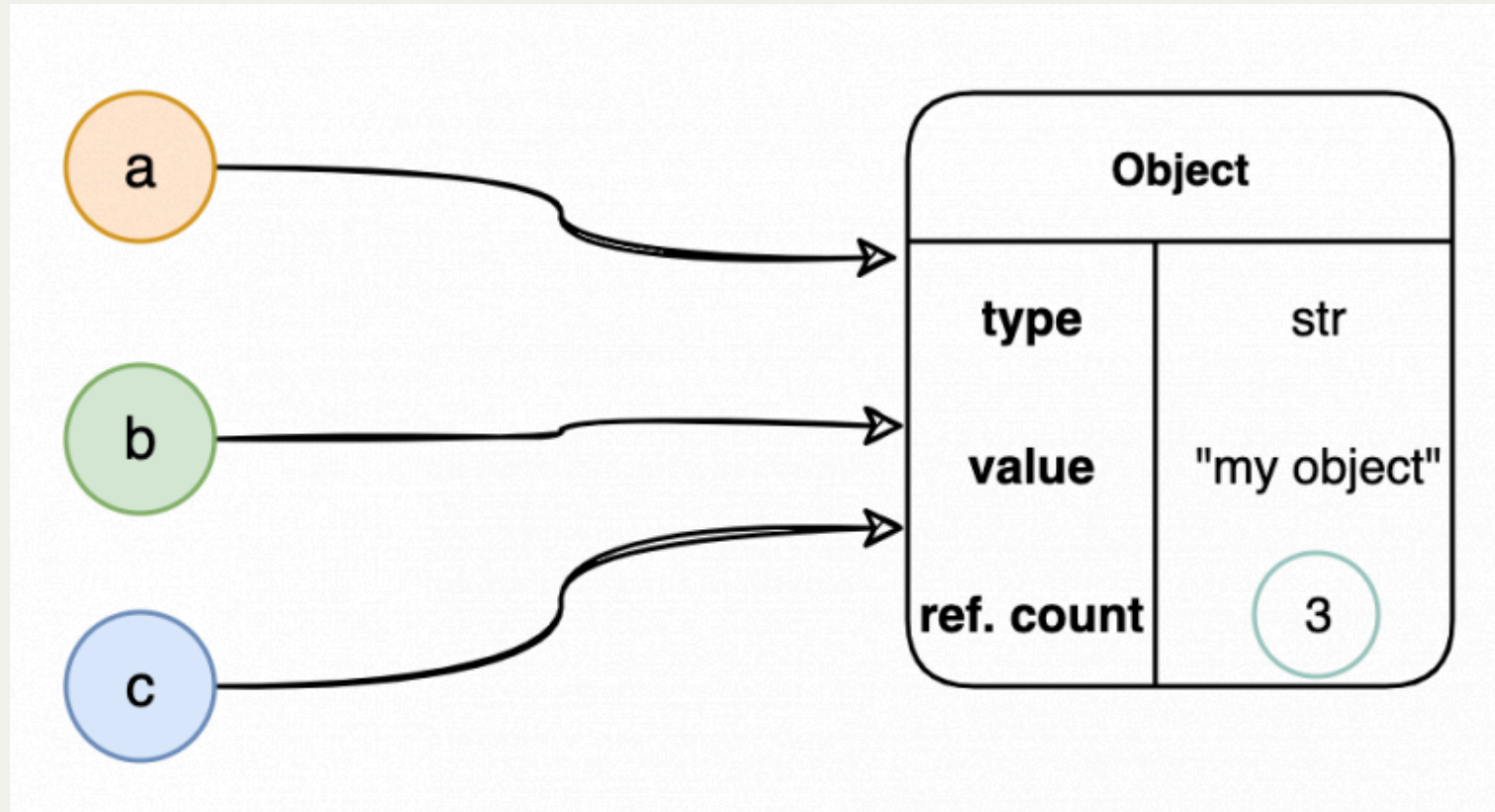
- Chosen for deterministic, immediate memory cleanup

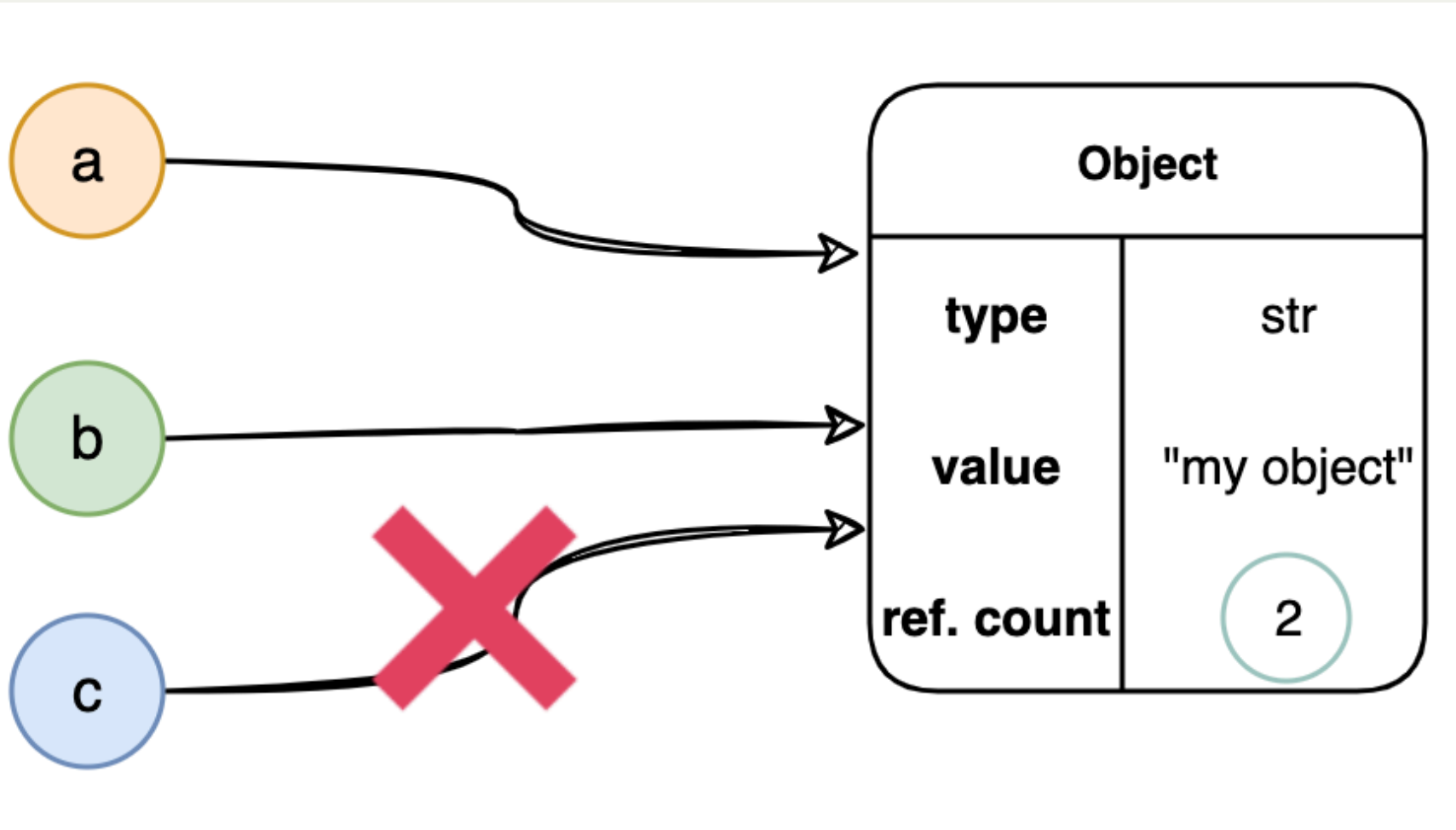
Reference Counting

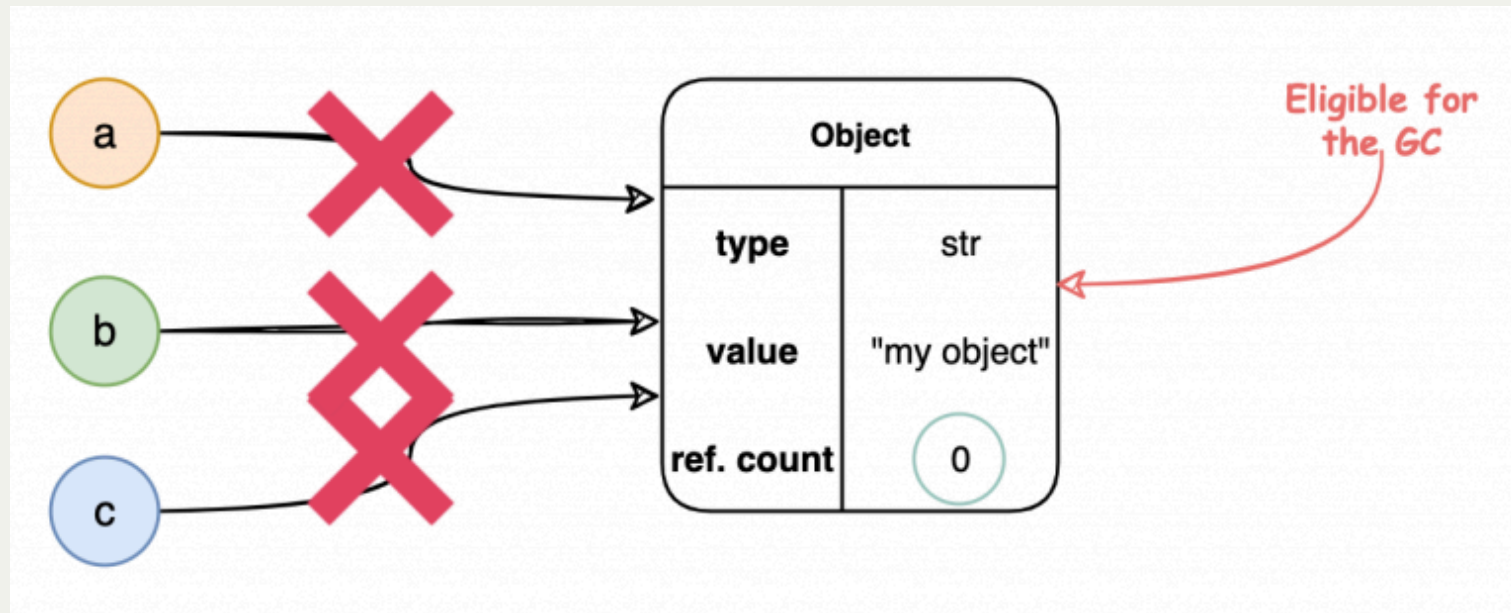
- Chosen for deterministic, immediate memory cleanup
- Reference count goes up with new references, down when deleted or reassigned

Reference Counting

- Chosen for deterministic, immediate memory cleanup
- Reference count goes up with new references, down when deleted or reassigned
- When it hits zero — the object is immediately freed







Why Reference Counting Is a Problem

Why Reference Counting Is a Problem

- Python tracks memory via reference counts

Why Reference Counting Is a Problem

- Python tracks memory via reference counts
- GIL protects those updates today

Why Reference Counting Is a Problem

- Python tracks memory via reference counts
- GIL protects those updates today
- Without GIL → race conditions

Reference Counting Fixes

- Biased Reference Counting
- Immortal Objects
- Deferred Reference Counting

Biased Reference Counting

Biased Reference Counting

- Most objects are accessed by just one thread

Biased Reference Counting

- Most objects are accessed by just one thread
- Optimized for single-threaded access

Biased Reference Counting

- Most objects are accessed by just one thread
- Optimized for single-threaded access
- Fast for owning thread

Biased Reference Counting

- Most objects are accessed by just one thread
- Optimized for single-threaded access
- Fast for owning thread
- Atomic ops only when needed

```
1 struct _object {
2     _PyObject_HEAD_EXTRA
3     uintptr_t ob_tid;           // owning thread id (4-8 bytes)
4     uint16_t __padding;        // reserved for future use (2 bytes)
5     PyMutex ob_mutex;          // per-object mutex (1 byte)
6     uint8_t ob_gc_bits;        // GC fields (1 byte)
7     uint32_t ob_ref_local;     // local reference count (4 bytes)
8     Py_ssize_t ob_ref_shared;  // shared reference count and state bits (4-8 bytes)
9     PyObject *ob_type;
10 };
```

```
1 struct _object {
2     _PyObject_HEAD_EXTRA
3     uintptr_t ob_tid;           // owning thread id (4-8 bytes)
4     uint16_t __padding;        // reserved for future use (2 bytes)
5     PyMutex ob_mutex;          // per-object mutex (1 byte)
6     uint8_t ob_gc_bits;        // GC fields (1 byte)
7     uint32_t ob_ref_local;      // local reference count (4 bytes)
8     Py_ssize_t ob_ref_shared;  // shared reference count and state bits (4-8 bytes)
9     PyObject *ob_type;
10 };
```

Immortal Objects

Immortal Objects

- Common objects are never deallocated

Immortal Objects

- Common objects are never deallocated
 - True, False, None, interned strings, small integers, etc.

Immortal Objects

- Common objects are never deallocated
 - True, False, None, interned strings, small integers, etc.
- Skip reference count updates

Immortal Objects

- Common objects are never deallocated
 - True, False, None, interned strings, small integers, etc.
- Skip reference count updates
- Reduce thread contention

Deferred Reference Counting

Deferred Reference Counting

- Skip reference count changes for hot objects

Deferred Reference Counting

- Skip reference count changes for hot objects
 - Reference counting on every access is inefficient

Deferred Reference Counting

- Skip reference count changes for hot objects
 - Reference counting on every access is inefficient
- GC handles cleanup later

Deferred Reference Counting

- Skip reference count changes for hot objects
 - Reference counting on every access is inefficient
- GC handles cleanup later
- Used for code, functions, modules

Garbage Collection (GC)

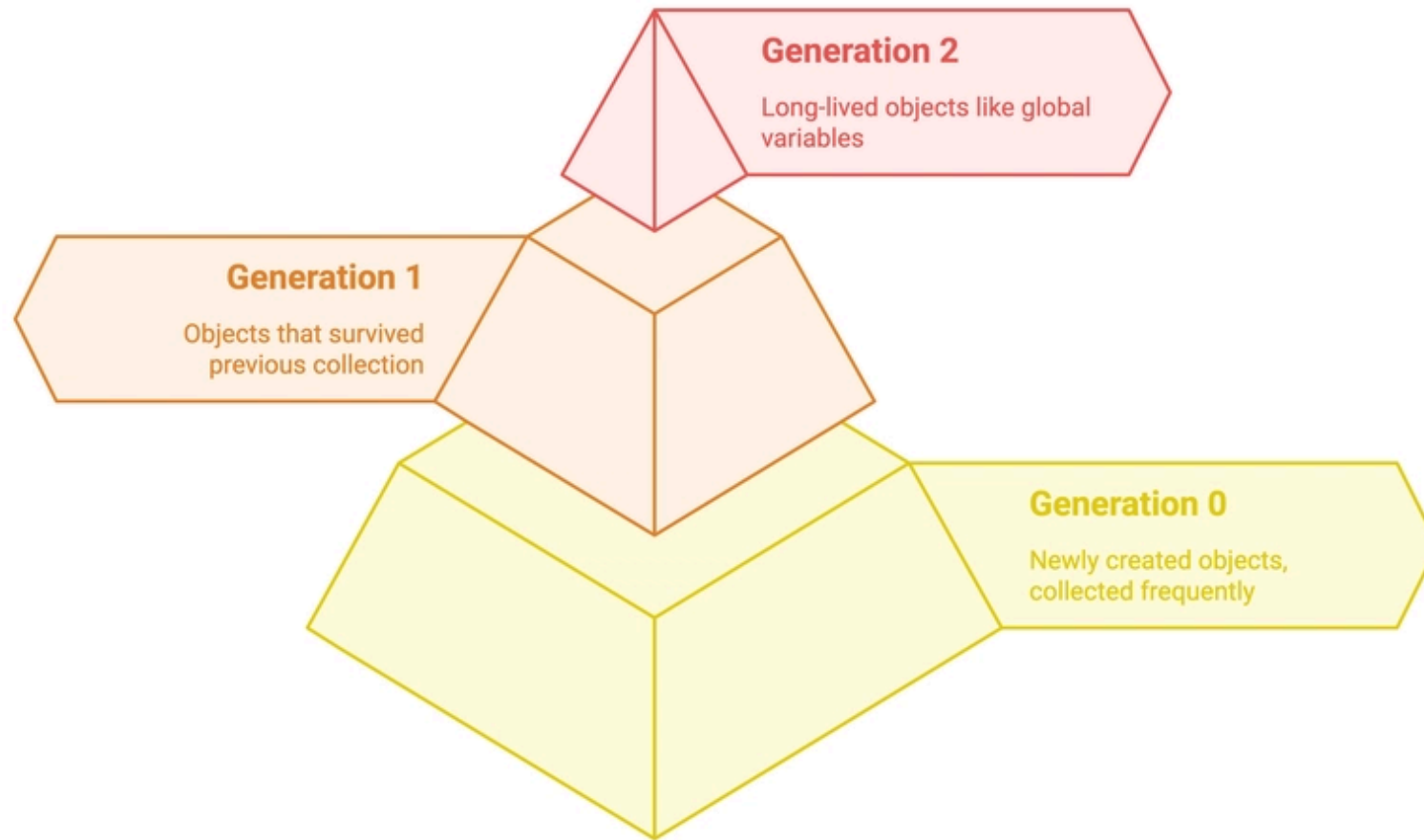
Garbage Collection (GC)

- Detects and collects **cyclic references** that reference counting misses

Garbage Collection (GC)

- Detects and collects **cyclic references** that reference counting misses
- Uses **generational GC**: objects are grouped by age; older ones are scanned less often

Object Lifecycle in Garbage Collection



GC without GIL

GC without GIL

- Can't rely on the GIL to **synchronize memory access** — GC must now be **explicitly thread-safe**

GC without GIL

- Can't rely on the GIL to **synchronize memory access** — GC must now be **explicitly thread-safe**
- “Stop-the-world” GC needed

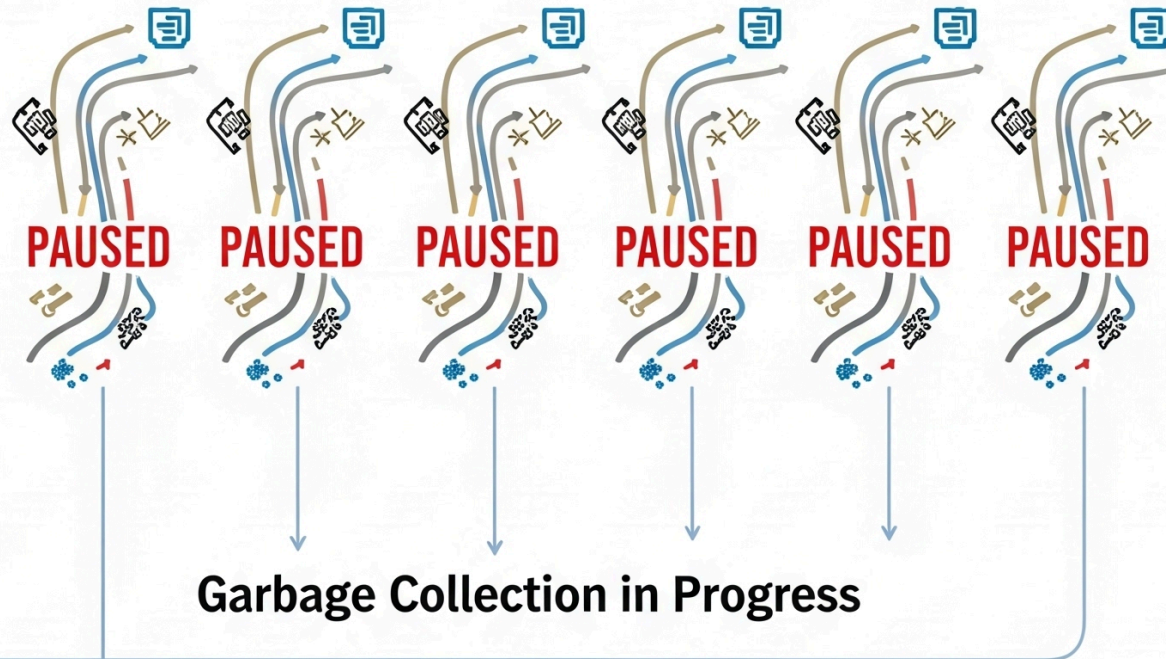
GC without GIL

- Can't rely on the GIL to **synchronize memory access** — GC must now be **explicitly thread-safe**
- “Stop-the-world” GC needed
 - All threads pause during collection to avoid race conditions

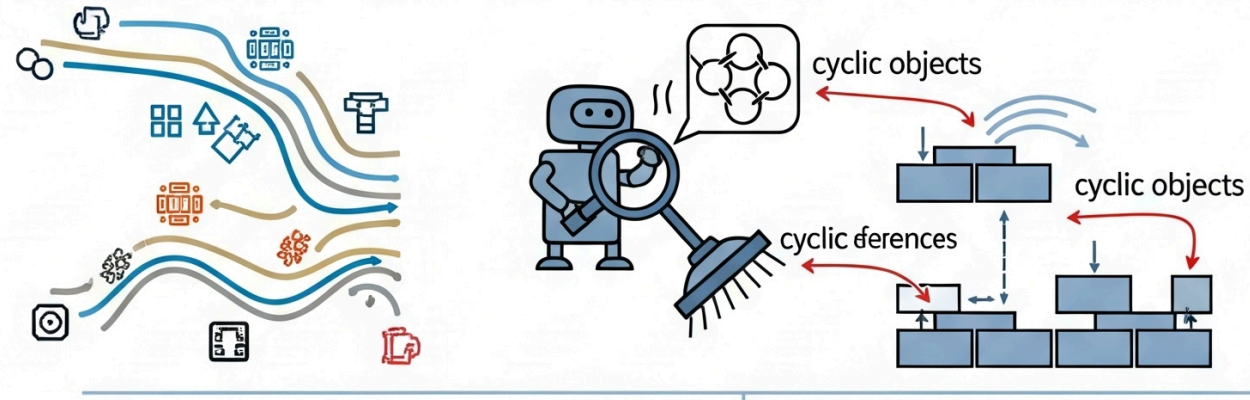
Normal Operation (Threads Active)



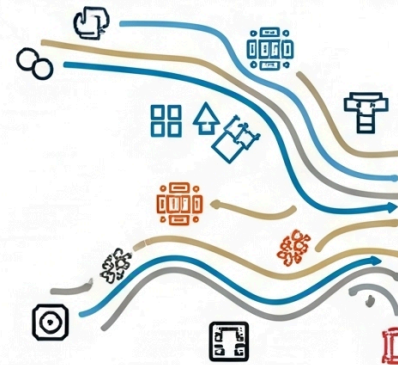
Garbage Collection Triggered



Garbage Collection in Progress



Threads Resumed



GC without GIL

GC without GIL

- Single-generation GC

GC without GIL

- Single-generation GC
 - Using a **single generation** avoids frequent pauses

GC without GIL

- Single-generation GC
 - Using a **single generation** avoids frequent pauses
 - **Generational GC is less critical** in Python — young objects are usually cleaned up early via **reference counting**

GC without GIL

- Single-generation GC
 - Using a **single generation** avoids frequent pauses
 - **Generational GC is less critical** in Python — young objects are usually cleaned up early via **reference counting**
- Integration with deferred and biased reference counting

Locking and atomic APIs

Locking and atomic APIs

- Global locks like the **GIL** add overhead but **no deadlocks** if only one lock exists

Locking and atomic APIs

- Global locks like the **GIL** add overhead but **no deadlocks** if only one lock exists
- Using **multiple locks** can cause **deadlocks** (e.g., double locks)

Locking and atomic APIs

- Global locks like the **GIL** add overhead but **no deadlocks** if only one lock exists
- Using **multiple locks** can cause **deadlocks** (e.g., double locks)
- Python uses **critical sections locked on individual objects**

Locking and atomic APIs

- Global locks like the **GIL** add overhead but **no deadlocks** if only one lock exists
- Using **multiple locks** can cause **deadlocks** (e.g., double locks)
- Python uses **critical sections locked on individual objects**
 - Lock acquired only when accessing that object

Locking and atomic APIs

- Global locks like the **GIL** add overhead but **no deadlocks** if only one lock exists
- Using **multiple locks** can cause **deadlocks** (e.g., double locks)
- Python uses **critical sections locked on individual objects**
 - Lock acquired only when accessing that object
 - Released if thread needs another lock or pauses (e.g., for blocking I/O)

```
1 struct _object {
2     _PyObject_HEAD_EXTRA
3     uintptr_t ob_tid;           // owning thread id (4-8 bytes)
4     uint16_t __padding;        // reserved for future use (2 bytes)
5     PyMutex ob_mutex;          // per-object mutex (1 byte)
6     uint8_t ob_gc_bits;        // GC fields (1 byte)
7     uint32_t ob_ref_local;      // local reference count (4 bytes)
8     Py_ssize_t ob_ref_shared;  // shared reference count and state bits (4-8 bytes)
9     PyObject *ob_type;
10 };
```

```
Py_BEGIN_CRITICAL_SECTION(a->ob_mutex);  
item = a->ob_item[i];  
Py_INCREF(item);  
Py_END_CRITICAL_SECTION(a->ob_mutex);
```

Locking and atomic APIs

Locking and atomic APIs

- **Read-write locks** are expensive and complex

Locking and atomic APIs

- **Read-write locks** are expensive and complex
 - Instead, Python prefers **try-fast approaches** with reference count checks

Locking and atomic APIs

- **Read-write locks** are expensive and complex
 - Instead, Python prefers **try-fast approaches** with reference count checks
 - Similar to **Read-Copy-Update (RCU)** pattern in Linux for efficient simple field access (e.g., `list[0]`)

Containers Thread-Safety

Containers Thread-Safety

- `list`, `set`, `dict`...

Containers Thread-Safety

- `list`, `set`, `dict`...
- Per-object locks replace GIL safety

Containers Thread-Safety

- `list`, `set`, `dict`...
- Per-object locks replace GIL safety
- Read operations may skip locks

Borrowed References

Borrowed References

- A **borrowed reference** is a pointer to a Python object you get **without** increasing its reference count

Borrowed References

- A **borrowed reference** is a pointer to a Python object you get **without** increasing its reference count
- To keep an object alive beyond immediate use, code must **convert borrowed to owned** by calling `Py_INCREF`

Borrowed References

Borrowed References

- Borrowed references were safe under the GIL
 - No other thread could mutate the object in between

Borrowed References

- Borrowed references were safe under the GIL
 - No other thread could mutate the object in between
- Now unsafe:
 - Another thread might free the object before `Py_INCREF`
 - Leads to **use-after-free** bugs

```
1 // 'item' is a borrowed reference returned by PyList_GetItem
2 PyObject *item = PyList_GetItem(list, idx);
3
4 // Another thread might free 'item' here before we increment its refcount
5
6 // Trying to increment the refcount to keep 'item' alive
7 // ✗ Unsafe: 'item' might be already freed, leading to use-after-free
8 Py_INCREF(item);
```

```
1 // 'item' is a borrowed reference returned by PyList_GetItem
2 PyObject *item = PyList_GetItem(list, idx);
3
4 // Another thread might free 'item' here before we increment its refcount
5
6 // Trying to increment the refcount to keep 'item' alive
7 // ✗ Unsafe: 'item' might be already freed, leading to use-after-free
8 Py_INCREF(item);
```

```
1 // 'item' is a borrowed reference returned by PyList_GetItem
2 PyObject *item = PyList_GetItem(list, idx);
3
4 // Another thread might free 'item' here before we increment its refcount
5
6 // Trying to increment the refcount to keep 'item' alive
7 // ✗ Unsafe: 'item' might be already freed, leading to use-after-free
8 Py_INCREF(item);
```

Atomic “Fetch” APIs (Slower)

New APIs return **owned references** safely in one step

```
//  Safe - Py_INCREF is done internally  
PyObject *item = PyList_FetchItem(list, idx);`
```

Optimistic Fast Access

Optimistic Fast Access

- Common list/dict accesses **avoid locks** for better speed and scalability

Optimistic Fast Access

- Common list/dict accesses **avoid locks** for better speed and scalability
- How it works:

Optimistic Fast Access

- Common list/dict accesses **avoid locks** for better speed and scalability
- How it works:
 - Atomically read slots and items

Optimistic Fast Access

- Common list/dict accesses **avoid locks** for better speed and scalability
- How it works:
 - Atomically read slots and items
 - Conditionally increment reference count (Py_INCREF) if safe

Optimistic Fast Access

- Common list/dict accesses **avoid locks** for better speed and scalability
- How it works:
 - Atomically read slots and items
 - Conditionally increment reference count (Py_INCREF) if safe
 - Verify no changes during access

Optimistic Fast Access

- Common list/dict accesses **avoid locks** for better speed and scalability
- How it works:
 - Atomically read slots and items
 - Conditionally increment reference count (Py_INCREF) if safe
 - Verify no changes during access
- If any check fails, **fallback** acquires a lock to ensure safety

Open Issues

Open Issues

- Still behind build flag

Open Issues

- Still behind build flag
- Library & package adoption

Open Issues

- Still behind build flag
- Library & package adoption
- Maintaining two runtimes

Open Issues

- Still behind build flag
- Library & package adoption
- Maintaining two runtimes
- Debugging & profiling gaps

Open Issues

- Still behind build flag
- Library & package adoption
- Maintaining two runtimes
- Debugging & profiling gaps
- Higher memory use

Open Issues

- Still behind build flag
- Library & package adoption
- Maintaining two runtimes
- Debugging & profiling gaps
- Higher memory use
- When is it stable?

Python's Future

Python's Future

- Python can finally use many cores at once

Python's Future

- Python can finally use many cores at once
- Big speedups for multi-threaded programs

Python's Future

- Python can finally use many cores at once
- Big speedups for multi-threaded programs
- New memory designs keep things safe

Python's Future

- Python can finally use many cores at once
- Big speedups for multi-threaded programs
- New memory designs keep things safe
- Still the same Python we know

Any Questions?

bit.ly/nogil

Yonatan Bitton
[@bityob](https://twitter.com/bityob)

linktr.ee/bityob

Sources

- PEP 703 – Making the Global Interpreter Lock Optional in CPython ([peps.python.org](https://peps.python.org/pep-0703/))
- PEP 779 – Criteria for supported status for free-threaded Python ([peps.python.org](https://peps.python.org/pep-0779/))
- Python Free-Threading Guide (py-free-threading.github.io)

Learn More

- Multithreaded Python without the GIL - presented by Sam Gross (youtube.com)
 - High-Performance Python: Faster Type Checking and Free Threaded Execution (youtube.com)
 - Python Docs How To - Python experimental support for free (docs.python.org)
-